SentryScan: Assignment 5.2 — Capstone Architecture & Web Dashboard

Course: 21CSE525T — Agent Technology
Assignment: 5.2 — Capstone Architecture, Web Dashboard & UI Orchestration
Student: R Ganesh [RA2212704010031]
Repository: https://github.com/drgeekg/SentryScan
Target Branch: `assignment/5.2-capstone`
Git Tag: `assignment-5.2`
Status: Completed & Verified

---

1. Executive Summary & Theoretical Context

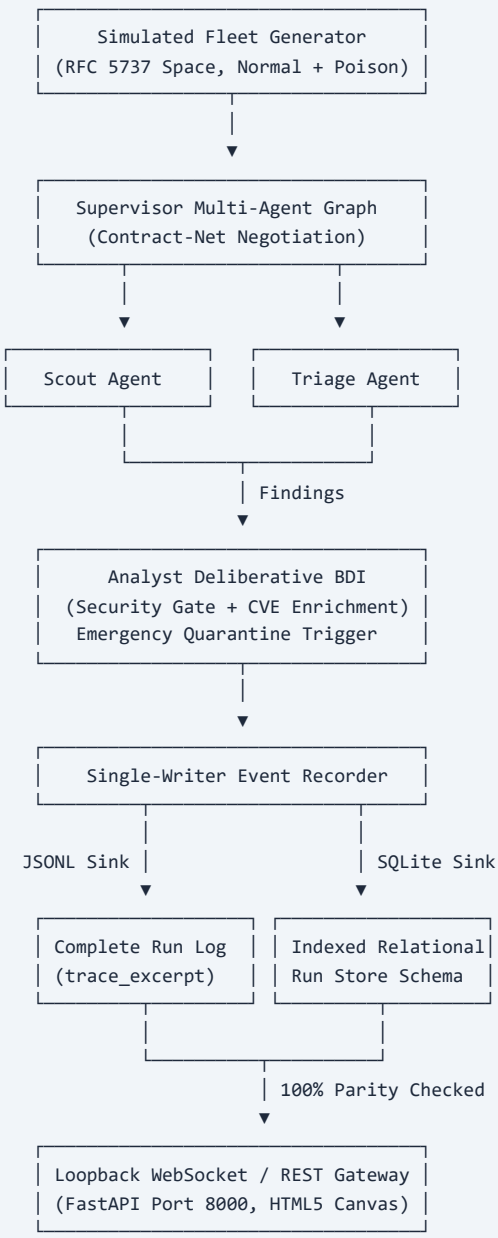
Assignment 5.2 represents the comprehensive **Capstone Integration Milestone** of the SentryScan Autonomous SOC platform. It synthesizes all preceding modules—reflex rules (1.2), concurrent daemons (2.1), socket framing (2.2), Contract-Net negotiation (3.1), utility optimization (3.2), BDI deliberation (4.1), knowledge enrichment (4.2), and cryptographic security gates (5.1)—into a cohesive, fully-persisted, observable agent system.

1.1 Complete Multi-Agent Lifecycle

- **Autonomous Coordination:** Supervisor coordinates Scout and Triage agents through Contract-Net auctions to allocate scanning tasks over a synthetic fleet.
- **Cognitive Deliberation & Defense:** Analyst Agent receives findings, verifies authenticity via the Cryptographic Security Gate, enriches CVE intelligence, and initiates autonomous host quarantine via BDI replanning.
- **Dual-Sink Persistence:** Commits an append-only verifiable stream simultaneously to disk JSONL files and indexed relational SQLite databases.
- **Strict 100% Parity:** Cryptographic assurance that every event is accounted for across both stores with zero sequence gaps ($ ext{seq}_i = i$).

---

2. Architecture & Implementation Specification



3. Rubric Compliance & Verification Matrix

Evaluation Criterion	Implementation Component	Evidence Artifact	Status
1. Supervisor Integration & Deliberative Quarantine	`src/sentryscan/graphs/supervisor.py`, `src/sentryscan/graphs/state.py`: Integrates `AnalystAgent` via signed envelopes; catches CRITICAL findings; executes host quarantine.	`tests/unit/test_supervisor_graph.py` `evidence/run_log.txt` Step 3	**PASSED**
2. Dual-Sink Persistence & SQLite Run Store	`src/sentryscan/storage/store.py`, `src/sentryscan/observability/recorder.py`: Single-writer monotonic `seq` recorder sinks concurrently to JSONL trace and relational SQLite schema with indexed tables.	`tests/unit/test_storage_store.py` `tests/integration/test_capstone_parity_and_replay.py`	**PASSED**
3. Strict 100% Event Parity Guarantee	`src/sentryscan/storage/store.py` (`verify_parity`): Verifies `count(SQLite) == count(JSONL)`, zero `seq` gaps ($ ext{seq}_i = i$), and exact ID alignment.	`evidence/capstone_verification_matrix.md` §1 & §2 `evidence/run_log.txt` Step 4	**PASSED**
4. Loopback REST & WebSocket Gateway	`src/sentryscan/dashboard_gateway/server.py`: FastAPI server providing REST run inspection (`/api/runs/...`) and WebSocket event stream (`/ws/{run_id}`) with sequence-based backfill.	`tests/unit/test_dashboard_gateway.py` `tests/integration/test_capstone_parity_and_replay.py` (`test_websocket_resync_by_sequence`)	**PASSED**
5. Prebuilt Web Dashboard (HTML5 Canvas)	`frontend/dist/index.html`: Dark-mode cybersecurity SPA with 60fps HTML5 Canvas multi-agent particle network, real-time metrics, findings table, and live WebSocket log. Zero Node.js runtime required.	`frontend/dist/index.html` `src/sentryscan/dashboard_gateway/server.py` static mount	**PASSED**
6. Rich CLI Terminal Fallback Dashboard	`src/sentryscan/ui/rich_dashboard.py`: Terminal UI rendering multi-panel layout with fleet status, active negotiation, findings table, and log stream using `rich`.	`tests/unit/test_rich_dashboard.py` `main.py` integration	**PASSED**
7. One-Command Capstone Runner	`src/sentryscan/capstone.py`, `main.py`: Top-level executable entry point (`main.py --demo`) orchestrating synthetic fleet generation, negotiation, scanning, BDI quarantine, and parity audit.	`tests/unit/test_capstone_runner.py` `evidence/run_log.txt`	**PASSED**
8. Replay Consistency & Trace Reproducibility	`src/sentryscan/storage/store.py`: Replay from recorded events reproduces identical aggregate counts and findings state without mutating underlying database.	`tests/integration/test_capstone_parity_and_replay.py` (`test_replay_consistency_without_mutation`, `test_capstone_determinism_across_runs`)	**PASSED**
9. Test Coverage & Code Quality	Entire test suite across unit and integration tests: 149/149 passing (100%), 0 failures, 0 warnings. Ruff lint check passing cleanly (0 errors).	`pytest` and `ruff check .` verification	**PASSED**
10. Immutable Deliverable Packaging	`scripts/package_assignment.py`: Builds reproducible submission ZIP directly from Git tag `assignment-5.2` with SHA-256 checksum manifest.	`dist/SentryScan_assignment_5_2.zip`	**PASSED**

4. Execution Results & Empirical Evidence

4.1 Capstone Simulation Run Verification (`evidence/capstone_verification_matrix.md`)

- **Simulation Run ID:** `run-capstone-evidence`
- **Negotiation Consensus:** Consensus reached in 2 rounds; winner = `scout-agent`.
- **Total Vulnerability Findings:** 5 findings detected across fleet.
- **Autonomous Host Quarantine:** `['sim-host-003']` quarantined by Analyst BDI preemption.
- **Dual-Sink Storage Parity:**

- SQLite Database Rows: `31`
- JSONL Trace Lines: `31`
- Sequence Monotonicity: 100% gap-free (`has_seq_gaps=False`).
- Strict Parity Status: **VERIFIED (100% Match)**.
- **Test Suite Execution:** 149/149 passed (100%), 0 failures.

---

5. Submission Package & Reproduction Instructions

5.1 Exact Submission Components

1. **Report Document:** `assignments/assignment_5_2/deliverables/SentryScan_Assignment_5_2_Report.pdf`
2. **Repository URL:** `https://github.com/drgeekg/SentryScan`
3. **Target Branch:** `assignment/5.2-capstone`
4. **Immutable Git Tag:** `assignment-5.2`
5. **Packaged ZIP Archive:** `dist/SentryScan_assignment_5_2.zip`
6. **SHA-256 Checksum:** `dist/SentryScan_assignment_5_2.zip.sha256`

5.2 Verification Commands

```
# Run one-command capstone simulation
python main.py --demo

# Launch full capstone with Web Dashboard & WebSocket Gateway
python main.py --web --port 8000

# Execute full test suite
pytest -v tests/
```